

ECE 408 Fall 2024 Final Project

Kaustubh Khulbe - kkhulbe2

December 2024

Contents

1	Files	2
2	Baseline	3
3	Req 0: Streams	5
3.1	Motivation	5
3.2	Integration	5
3.3	Code	5
3.4	Analysis	9
3.5	References	9
4	Req 1: Tensor Cores	10
4.1	Motivation	10
4.2	Integration	10
4.3	Code	10
4.4	Analysis	14
4.5	References	14
5	Req 2: Kernel Fusion	16
5.1	Motivation	16
5.2	Integration	16
5.3	Code	16
5.4	Analysis	19
5.5	References	20
6	Op 0: Constant Memory	21
6.1	Motivation	21
6.2	Integration	21
6.3	Code	21
6.4	Analysis	22
6.5	References	22

7	Op 1: Restrict	23
7.1	Motivation	23
7.2	Integration	23
7.3	Code	23
7.4	Analysis	24
7.5	References	24
8	Op 4: cuBLAS	25
8.1	Motivation	25
8.2	Integration	25
8.3	Code	25
8.4	Analysis	26
8.5	References	26
9	Op 5: FP16	27
9.1	Code	27
9.2	Analysis	28
10	Conclusion	29

1 Files

<https://drive.google.com/drive/folders/1B39oQwOXC2BTbx329wgrSe8EGsNE95Bf?usp=sharing>

2 Baseline

Attached is the baseline performance of the kernel from *PM1* and *PM2*. We will reference these throughout to gauge performance improvements in certain aspects.

Compute (SM) Throughput [%]	36.59	Duration [ms]	159.94
Memory Throughput [%]	36.59	Elapsed Cycles [cycle]	208754885
L1/TEX Cache Throughput [%]	36.69	SM Active Cycles [cycle]	208117932.29
L2 Cache Throughput [%]	2.95	SM Frequency [Ghz]	1.30
DRAM Throughput [%]	2.23	DRAM Frequency [Ghz]	7.25

Figure 1: PM1 SOL Baseline: Conv Forward

Compute (SM) Throughput [%]	4.58	Duration [ms]	78.78
Memory Throughput [%]	70.82	Elapsed Cycles [cycle]	102504675
L1/TEX Cache Throughput [%]	19.68	SM Active Cycles [cycle]	102822842.06
L2 Cache Throughput [%]	34.89	SM Frequency [Ghz]	1.30
DRAM Throughput [%]	70.82	DRAM Frequency [Ghz]	7.22

Figure 2: PM2 SOL Baseline: Matrix Unroll

Compute (SM) Throughput [%]	96.57	Duration [ms]	37.24
Memory Throughput [%]	96.57	Elapsed Cycles [cycle]	48595988
L1/TEX Cache Throughput [%]	96.58	SM Active Cycles [cycle]	48587890.98
L2 Cache Throughput [%]	14.29	SM Frequency [Ghz]	1.30
DRAM Throughput [%]	53.88	DRAM Frequency [Ghz]	7.24

Figure 3: PM2 SOL Baseline: Matrix Multiplication

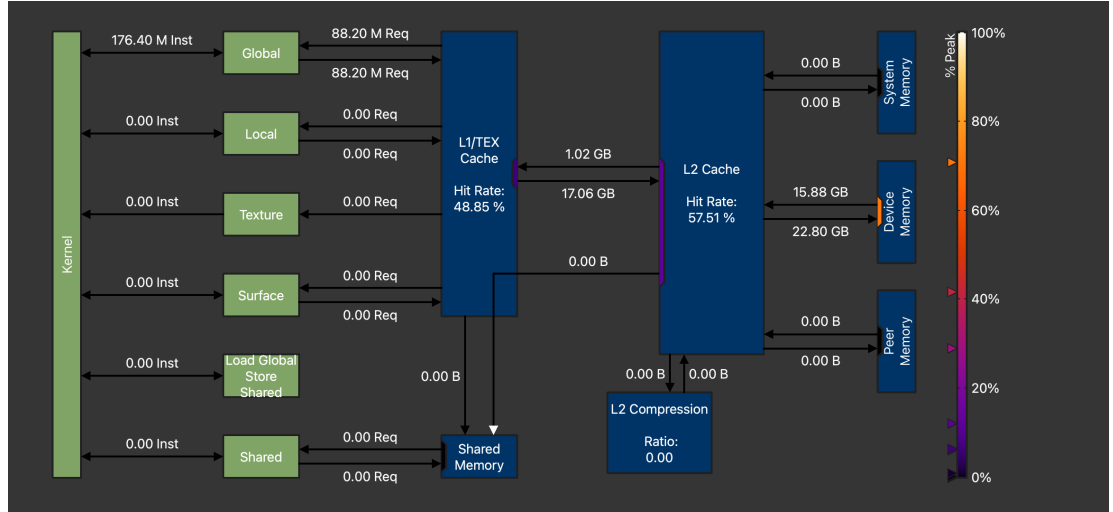


Figure 4: PM2 Memory Baseline: Matrix Unroll

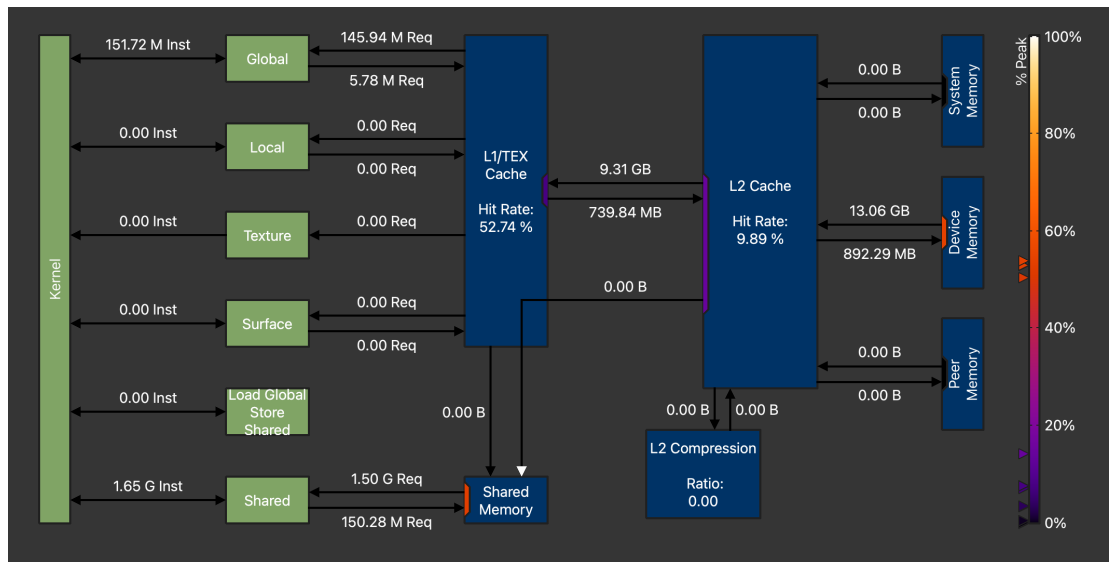


Figure 5: PM2 Memory Baseline: Matrix Multiplication

3 Req 0: Streams

https://drive.google.com/drive/folders/1POZNd8I3OqVbr_FxBmuUsu7uRC2-SEAG?usp=sharing

3.1 Motivation

There are typically two primary operations we do. Moving data around, and operating on the data. In other words, *memcpy* and kernel execution. In order to overlap these executions, CUDA has an API: the *streams* API. On a high level, the CUDA programming model has two engines that will separately run the compute and memory operations, allowing seamless overlapping between costly data transfers and kernel execution. The following code expands the PM2 implementation to incorporate streams. *Note:* everything is moved into the prologue for simplicity. As such, optimes are no longer a sufficient measurement.

My approach was to have three streams, and a for loop through all the batches, having each stream responsible for a chunk of batches (kind of like MT2!). We begin with the declarations and copying of memory. The number of batches is dynamically set up based on the batch size. If there is a large amount of batches, we split amongst larger batches so kernel launch overhead does not become too dominant.

3.2 Integration

The streams API and software implemented synergizes with most, if not all optimizations. At its core, every optimization still needs to copy host memory, launch kernels, and write to host memory from device memory. This means that there is a use case for streams.

3.3 Code

What's about to follow looks like a big dump of code, but it's just set up code. This includes the streams, variables, and initial memcpys to set up the data.

```
cudaStream_t streams[3];
cudaStreamCreate(&streams[0]);
cudaStreamCreate(&streams[1]);
cudaStreamCreate(&streams[2]);

int num_batches = ceil(Batch / 50);

const int Height_out = Height - K + 1;
const int Width_out = Width - K + 1;
```

```

const int Height_unrolled = Channel * K * K;
const int Width_unrolled = num_batches * Height_out * Width_out;

const int Height_conv_filters = Map_out;
const int Width_conv_filters = Channel * K * K;

const int Height_matmul = Map_out;
const int Width_matmul = num_batches * Height_out * Width_out;

/*
   Mask: (M_out) x (C * K * K)
   Unrolled matrix: (C * K * K) x (Batch * H_out * W_out)
   Matmul matrix: (M_out) * (Batch * H_out * W_out)
*/

float *d_mask0, *d_unroll_in0, *d_unroll0, *d_matmul0, *d_out0;
float *d_mask1, *d_unroll_in1, *d_unroll1, *d_matmul1, *d_out1;
float *d_mask2, *d_unroll_in2, *d_unroll2, *d_matmul2, *d_out2;

float *host_input_pinned, *host_mask_pinned;
cudaMallocHost( (void**) &host_input_pinned,
    Batch * Channel * Height * Width * sizeof(float));
cudaMallocHost( (void**) &host_mask_pinned,
    Map_out * Channel * K * K * sizeof(float));
cudaHostRegister((void*) host_output,
    Batch * Map_out * Height_out * Width_out * sizeof(float), cudaHostRegisterDefault);

memcpy(host_input_pinned, host_input, Batch * Channel * Height * Width * sizeof(float));
memcpy(host_mask_pinned, host_mask, Map_out * Channel * K * K * sizeof(float));

cudaMalloc((void**)&d_mask0, (size_t) Map_out * Channel * K * K * sizeof(float));
cudaMalloc((void**)&d_mask1, (size_t) Map_out * Channel * K * K * sizeof(float));
cudaMalloc((void**)&d_mask2, (size_t) Map_out * Channel * K * K * sizeof(float));

cudaMalloc((void**)&d_unroll_in0,
    num_batches * Channel * Height * Width * sizeof(float));
cudaMalloc((void**)&d_unroll_in1,
    num_batches * Channel * Height * Width * sizeof(float));
cudaMalloc((void**)&d_unroll_in2,
    num_batches * Channel * Height * Width * sizeof(float));

cudaMalloc((void**)&d_unroll0,
    (size_t) Channel*K*K * num_batches * Height_out * Width_out * sizeof(float));
cudaMalloc((void**)&d_unroll1,
    (size_t) Channel*K*K * num_batches * Height_out * Width_out * sizeof(float));
cudaMalloc((void**)&d_unroll2,

```

```

    (size_t) Channel*K*K * num_batches * Height_out * Width_out * sizeof(float));

cudaMalloc((void**)&d_matmul0,
    (size_t) Map_out * num_batches * Height_out * Width_out * sizeof(float));
cudaMalloc((void**)&d_matmul1,
    (size_t) Map_out * num_batches * Height_out * Width_out * sizeof(float));
cudaMalloc((void**)&d_matmul2,
    (size_t) Map_out * num_batches * Height_out * Width_out * sizeof(float));

cudaMalloc( (void**) &d_out0,
    Map_out * Height_out * (num_batches * Width_out) * sizeof(float));
cudaMalloc( (void**) &d_out1,
    Map_out * Height_out * (num_batches * Width_out) * sizeof(float));
cudaMalloc( (void**) &d_out2,
    Map_out * Height_out * (num_batches * Width_out) * sizeof(float));

cudaMemcpyAsync(d_mask0, host_mask_pinned,
    Map_out * Channel * K * K * sizeof(float), cudaMemcpyHostToDevice, streams[0]);
cudaMemcpyAsync(d_mask1, host_mask_pinned,
    Map_out * Channel * K * K * sizeof(float), cudaMemcpyHostToDevice, streams[1]);
cudaMemcpyAsync(d_mask2, host_mask_pinned,
    Map_out * Channel * K * K * sizeof(float), cudaMemcpyHostToDevice, streams[2]);

cudaStreamSynchronize(streams[0]);
cudaStreamSynchronize(streams[1]);
cudaStreamSynchronize(streams[2]);

float* masks[3] = {d_mask0, d_mask1, d_mask2};
float* unroll_ins[3] = {d_unroll_in0, d_unroll_in1, d_unroll_in2};
float* unrolls[3] = {d_unroll0, d_unroll1, d_unroll2};
float* matmuls[3] = {d_matmul0, d_matmul1, d_matmul2};
float* outs[3] = {d_out0, d_out1, d_out2};

int num_chunks = ceil(1.0 * Batch / num_batches);

```

Essentially, we create an array of three streams. For each of these streams, we create the necessary device memory required to process *num_chunks* batches.

```

for (int i = 0; i < num_chunks; ++i) {
    cudaStream_t stream = streams[i % 3];
    float* mask = masks[i%3];
    float* unroll_in = unroll_ins[i%3];
    float* unroll = unrolls[i%3];
    float* matmul = matmuls[i%3];
    float* out = outs[i%3];

    int current_num_batches;

```

```

if ((i+1) * num_batches > Batch) current_num_batches = Batch - i * num_batches;
else current_num_batches = num_batches;

int Width_unrolled = current_num_batches * Height_out * Width_out;
int Width_matmul = current_num_batches * Height_out * Width_out;

cudaMemcpyAsync(unroll_in,
    host_input + i*num_batches*Channel*Height*Width,
    current_num_batches * Channel * Height * Width * sizeof(float),
    cudaMemcpyHostToDevice, stream);

dim3 unroll_block(UNROLL_TILE_WIDTH, UNROLL_TILE_WIDTH, UNROLL_TILE_WIDTH);
dim3 unroll_grid(ceil(1.0 * Width_out / UNROLL_TILE_WIDTH),
    ceil(1.0 * Height_out / UNROLL_TILE_WIDTH),
    ceil(1.0 * Channel * current_num_batches / UNROLL_TILE_WIDTH));

matrix_unrolling_kernel<<<unroll_grid, unroll_block, 0, stream>>>
    (unroll_in, unroll, current_num_batches, Channel, Height, Width, K);

dim3 matmul_block(TILE_WIDTH, TILE_WIDTH, 1);
dim3 matmul_grid(ceil(1.0 * Width_matmul / TILE_WIDTH),
    ceil(1.0 * Height_matmul / TILE_WIDTH), 1);

matrixMultiplyShared<<<matmul_grid, matmul_block, 0, stream>>>
    (mask, unroll, matmul, Height_conv_filters,
    Width_conv_filters, Height_unrolled, Width_unrolled, Height_matmul, Width_matmul);

const int out_image_size = Height_out * Width_out;
dim3 permute_kernel_grid_dim((out_image_size - 1) / BLOCK_SIZE + 1,
    current_num_batches, 1);

matrix_permute_kernel<<<permute_kernel_grid_dim, BLOCK_SIZE, 0, stream>>>
    (
        matmul, out, Map_out, current_num_batches, out_image_size
    );

cudaMemcpyAsync((void*) host_output + i * num_batches *
    Map_out*Height_out*Width_out*sizeof(float), out,
    Map_out * Height_out * (current_num_batches * Width_out) * sizeof(float),
    cudaMemcpyDeviceToHost, stream);
}

```

Though appears dense, in principle the for loop is straight forward. We gather the corresponding device memory arrays and stream corresponding to the correct iteration. That is the first few lines. From there, if the batches

left to process is less than *num_batches*, we cap it as needed. We then load in the data, perform the unroll, matmul, permute, and write it back. This is done many times, and features a round robin scheduler to allocate compute and memory tasks to the streams. There is also some logic to ensure that the current batches we are about to process do not exceed the total amount of batches.

3.4 Analysis

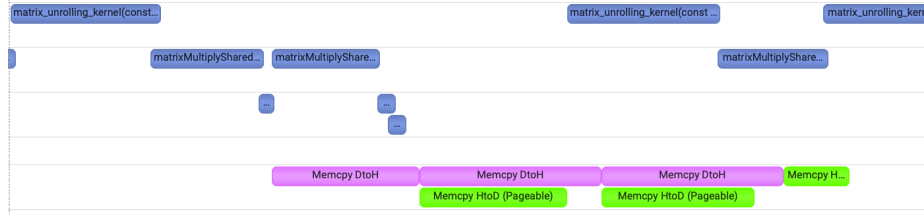


Figure 6: Nsight Systems: Req 0

From the figure, we see the kernel execution and memory copying engines working much better together, with some kernels executed entirely while a mem-copy for another stream was occurring. These overlapping memory and compute actions suggest successful streaming.

The software mostly performed as expected, albeit the clumped together copy's was a bit surprising. However, it is reasonable as we copy in the beginning and copy in the end of our loops, so along consecutive iterations we get groups copies with kernel execution in the middle.

We have verified with Nsight profiling that our streaming implementation is correct. A natural extension of this would be to support more streams, or possibly tuning the batch each stream processes.

3.5 References

1. <https://developer.nvidia.com/blog/how-overlap-data-transfers-cuda-cc/>
2. ECE 408 Fa24 Lecture 18: Tasks Parallelism and Asynchronous Data Transfer

4 Req 1: Tensor Cores

https://drive.google.com/drive/folders/1NTtdx4af7AZmhu5f6F1Tjo-cvJ60R24_?usp=sharing

4.1 Motivation

Matrix multiplication is an incredibly important task, and moreover a time consuming task. Nvidia GPUs now provide tensor cores, which are designated hardware to perform the *matrix multiply and accumulate* command. In effect, perform $D = AB + C$ as fast as possible.

Briefly skimming the hardware, Nvidia built a 16×16 matrix multiplier and accumulator via adders and multipliers. Then, from the register file, there lies a routing network to correctly set up the data for the adding and multiplying units, and outputs the tiled 16×16 multiplication. (Nvidia offers other tiled sizes, but I used 16).

In principle, we can now do a lot of computation on the GPU cores, transfer the GEMM to the tensor cores for high speed computation, and gather it and continue kernel execution.

4.2 Integration

This code also seamlessly integrates and synergies with the other optimizations. You would just need to swap out the basic matrix multiplication kernel we had in PM2 with the tensor core kernel.

4.3 Code

Before detailing my software, a couple considerations must be noted to explain design decisions.

1. A tensor core operation must require a tile size of at most 16×16
2. The wmma api applies on a *warp* level
3. The input are *half* types, while the output is a *float* type (this is due to hardware implementations)

This motivates the following grid and block sizes. We essentially just tile the matrix with 16×16 tiles.

```
dim3 matmul_block(16, 16, 1);
dim3 matmul_grid(ceil(1.0 * Width_matmul / TILE_WIDTH),
                 ceil(1.0 * Height_matmul / TILE_WIDTH), 1);

matrixMultiplyTensor<<<matmul_grid, matmul_block>>>
```

```
(device_mask, unrolled_matrix,
matmul_output, Height_conv_filters,
Width_conv_filters, Height_unrolled,
Width_unrolled, Height_matmul, Width_matmul);
```

We begin the kernel with some initialization. These are some useful numbers like the block indices, row/cols, and thread indices along with the fragments needed for the wmma API. We also initialize tiled matrices (explanation will follow).

```
int by = blockIdx.y, bx = blockIdx.x, ty = threadIdx.y, tx = threadIdx.x;
int row = by * TILE_WIDTH + ty, col = bx * TILE_WIDTH + tx;

wmma::fragment<wmma::matrix_a,
    WMMA_M, WMMA_N, WMMA_K, half, wmma::row_major> a_frag;
wmma::fragment<wmma::matrix_b,
    WMMA_M, WMMA_N, WMMA_K, half, wmma::row_major> b_frag;
wmma::fragment<wmma::accumulator,
    WMMA_M, WMMA_N, WMMA_K, float> acc_frag;

wmma::fill_fragment(acc_frag, 0.0f);

__shared__ __align__(32) half a_pad[256];
__shared__ __align__(32) half b_pad[256];
__shared__ __align__(32) float c_pad[256];
```

We then iterate along the K dimension of the matrices like in traditional tiled matrix multiplication, and perform the following computations. We load the corresponding elements of A and B into their respective fragments, **padding them with 0**. This is done because the wmma API requires 16×16 tiles, so we have to pad in case our matrices do not satisfy that. We then sync threads before performing the multiplication.

The following if condition, checking if $ty \cdot 16 + tx < 32$ was caught after hours of debugging (seriously...!). We have spawned a 16×16 block, which means there are 8 warps. This means we are over computing, as all warps would have executed the wmma commands. Thus, we have a condition to make sure only the first warp does this. *Note*, there is no warp divergence because *all* 7 warps do nothing, while *all* of the first warp executes the wmma.

```
if (row < numRows && tileId * TILE_WIDTH + tx < numAColumns) {
    a_pad[ty*16 + tx] =
        __float2half(A[(size_t)
            row * numAColumns + tileId * TILE_WIDTH + tx]);
} else a_pad[ty*16 + tx] = __float2half(0.0);

if (col < numBColumns && tileId * TILE_WIDTH + ty < numBRows) {
```

```

        b_pad[ty*16 + tx] =
            __float2half(B[((size_t)
                tileId * TILE_WIDTH + ty) * numBColumns + col]);
    } else b_pad[ty*16 + tx] = __float2half(0.0);

    __syncthreads();

    if (ty*16+tx < 32) {
        // Load the inputs
        wmma::load_matrix_sync(a_frag, a_pad, 16);
        wmma::load_matrix_sync(b_frag, b_pad, 16);

        // Perform the matrix multiplication
        wmma::mma_sync(acc_frag, a_frag, b_frag, acc_frag);
    }

    __syncthreads();

```

After the loop, we store the matrix (with the same warp condition), and write to C .

```

    if (ty*16+tx < 32) store_matrix_sync(c_pad, acc_frag, 16, wmma::mem_row_major);

    __syncthreads();

    // Store the output
    if (row < numCRows && col < numCColumns) {
        C[ row * numCColumns + col ] = c_pad[ty*16 + tx];
    }

```

Attached is the full Tensor Core Matrix Multiplication Kernel.

```

__global__ void matrixMultiplyTensor(const float *A, const float *B, float *C,
                                     int numRows, int numAColumns,
                                     int numBRows, int numBColumns,
                                     int numCRows, int numCColumns)
{
    int by = blockIdx.y, bx = blockIdx.x, ty = threadIdx.y, tx = threadIdx.x;
    int row = by * TILE_WIDTH + ty, col = bx * TILE_WIDTH + tx;

    wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::row_major> b_frag;
    wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;

    wmma::fill_fragment(acc_frag, 0.0f);

    __shared__ __align__(32) half a_pad[256];

```

```

__shared__ __align__(32) half b_pad[256];
__shared__ __align__(32) float c_pad[256];

float val = 0.0f;

for (int tileId = 0; tileId < (numAColumns - 1) / TILE_WIDTH + 1; tileId++) {
    // wmma::fill_fragment(acc_frag, 0.0f);
    if (row < numRows && tileId * TILE_WIDTH + tx < numAColumns) {
        a_pad[ty*16 + tx] =
            __float2half(A[(size_t)
                row * numAColumns + tileId * TILE_WIDTH + tx]);
    } else a_pad[ty*16 + tx] = __float2half(0.0);

    if (col < numBColumns && tileId * TILE_WIDTH + ty < numRows) {
        b_pad[ty*16 + tx] =
            __float2half(B[(size_t)
                tileId * TILE_WIDTH + ty) * numBColumns + col]);
    } else b_pad[ty*16 + tx] = __float2half(0.0);

    __syncthreads();

    if (ty*16+tx < 32) {
        // Load the inputs
        wmma::load_matrix_sync(a_frag, a_pad, 16);
        wmma::load_matrix_sync(b_frag, b_pad, 16);

        // Perform the matrix multiplication
        wmma::mma_sync(acc_frag, a_frag, b_frag, acc_frag);
    }

    __syncthreads();
}

if (ty*16+tx < 32) store_matrix_sync(c_pad, acc_frag, 16, wmma::mem_row_major);

__syncthreads();

// Store the output
if (row < numRows && col < numCColumns) {
    C[ row * numCColumns + col ] = c_pad[ty*16 + tx];
}
}

```

GPU Speed Of Light Throughput			
High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.			
Compute (SM) Throughput [%]	44.91	Duration [ms]	30.93
Memory Throughput [%]	77.04	Elapsed Cycles [cycle]	40351060
L1/TEX Cache Throughput [%]	38.94	SM Active Cycles [cycle]	40353267.32
L2 Cache Throughput [%]	23.26	SM Frequency [Ghz]	1.30
DRAM Throughput [%]	77.04	DRAM Frequency [Ghz]	7.24

Figure 7: SOL Analysis: Req 1

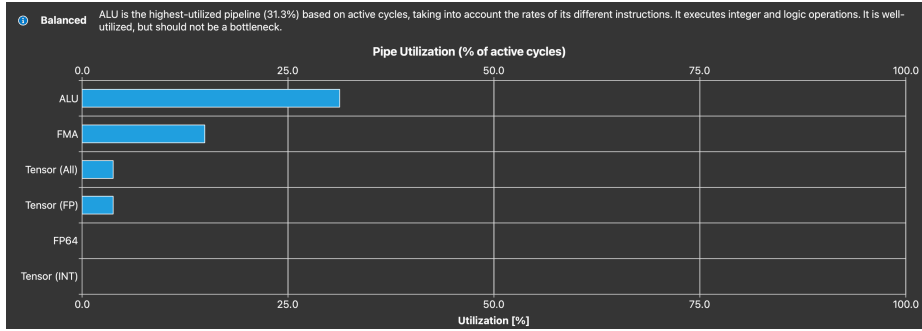


Figure 8: Compute Analysis: Req 1

4.4 Analysis

We notice an improvement in execution time with the tensor cores. At the best case, we notice an improvement of 45% for the second matrix multiplication kernel, and an improvement of 5% for the first matrix multiplication kernel. Examining the workload, this marginal improvement in the second kernel seems to be from high miss rates on the L2 cache, implying frequent DRAM access. However, if we look at the actual compute analysis, we can see that the tensor cores are being utilized, meaning we are actually sending fragments to the cores and computing with them.

This analysis conforms with our beliefs as it indicates increased tensor core usage, indicating CUDA correctly using the tensor cores for the matrix multiplication algorithm.

In general, we have higher memory throughput for this kernel compared to the initial matrix multiply kernel, implying that this is a memory bound kernel. This would explain why it only performs marginally better for larger workloads, as the memory throughput balloons the runtime.

4.5 References

1. <https://developer.nvidia.com/blog/programming-tensor-cores-cuda-9>
2. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html> warp-

matrix-functions

3. <https://github.com/NVIDIA-developer-blog/code-samples/tree/master/posts/tensor-cores>

5 Req 2: Kernel Fusion

<https://drive.google.com/drive/folders/1UxboxOp-u-uxmBI7z90g-nEBISmgODY3?usp=sharing>

5.1 Motivation

Launching CUDA kernels has overhead in it, in particular if the workload within the kernel is not significant. To remedy this, one strategy we can use is kernel fusion. In essence, we combine multiple kernels into one, to minimize launch overhead.

The second strategy we implemented in this requirement is dynamic loop unrolling. Rather than fully unrolling the convolution, we unroll only what we need, *when we need it*. This would have less overhead as it is less stores in global memory, and in general working with smaller sized arrays.

5.2 Integration

Updating our code from PM2 requires many modifications, including changing a lot of the kernels/deleting them. This also makes it a bit more tedious to synergize with the previous optimizations, as it moves a lot of the logic into one kernel, which will cause us to modify a lot more code. However, it's very doable, and I did play around with that for my stacked final submission.

5.3 Code

To implement this, we will have each thread compute one value in the output matrix. Thus, these are the block and grid dimensions we use.

```
dim3 blockDim(TILE_WIDTH, TILE_WIDTH);
dim3 gridDim(ceil(1.0 * Batch * Height_out * Width_out / TILE_WIDTH),
             ceil(1.0 * Map_out / TILE_WIDTH));
```

```
unroll_and_multiply_kernel<<<gridDim, blockDim>>>(device_input,
            device_output, device_mask,
            Batch, Channel, Height,
            Width, K, Map_out);
```

Within the actual kernel, we first compute which *row* and *col* we are on with

```
int row = threadIdx.y + blockIdx.y * TILE_WIDTH;
int col = threadIdx.x + blockIdx.x * TILE_WIDTH;
int tx = threadIdx.x;
int ty = threadIdx.y;
```

We have also encoded b, h, w into the launch configurations and can extract them the following way


```

int b = col / image_size;
int h = (col % image_size) / Width_out; // height index
int w = (col % image_size) % Width_out; // width index

```

We then iterate in tiles along the k dimension of our matrix, with size $C \cdot K^2$. The following code correctly loads M and N , the shared memory tiles.

```

if (row < Map_out && i * TILE_WIDTH + tx < Channel*K*K) {
    N[ty][tx] = mask[(size_t) row * Channel*K*K + i * TILE_WIDTH + tx];
} else N[ty][tx] = 0.0f;

int tile_pos = i * TILE_WIDTH + ty;

int c = tile_pos / (K*K);
int p = (tile_pos % (K*K)) / K;
int q = (tile_pos % (K*K)) % K;

if (b < Batch && c < Channel && h < Height_out && w < Width_out) {
    M[ty][tx] = in_4d(b, c, h + p, w + q);
} else M[ty][tx] = 0.0f;

```

From there, we do a standard matrix multiplication with

```

if (row < Map_out && col < numRows) {
    for (int k = 0; k < TILE_WIDTH; k++) {
        acc += N[ty][k] * M[k][tx];
    }
}

```

And at the very end, we write to the output if we are in bounds.

```

if (row < Map_out && col < numRows) {
    output[b * Map_out * image_size + row * image_size + h * Width_out + w] = acc;
}

```

The entire kernel is attached below.

```

__global__ void unroll_and_multiply_kernel(const float *input,
                                            float *output,
                                            const float *mask,
                                            const int Batch,
                                            const int Channel,
                                            const int Height,
                                            const int Width,
                                            const int K,
                                            const int Map_out) {

    #define in_4d(i3, i2, i1, i0) \

```

```

        input[(i3) * (Channel * Height * Width) + (i2) * \
        (Height * Width) + (i1) * (Width) + i0]

const int Height_out = Height - K + 1;
const int Width_out = Width - K + 1;

int numRows = Batch * Height_out * Width_out;
int numCols = Channel * K * K;

int row = threadIdx.y + blockIdx.y * TILE_WIDTH;
int col = threadIdx.x + blockIdx.x * TILE_WIDTH;
int tx = threadIdx.x;
int ty = threadIdx.y;

int image_size = Height_out * Width_out;

int b = col / image_size;
int h = (col % image_size) / Width_out; // height index
int w = (col % image_size) % Width_out; // width index

__shared__ float N[TILE_WIDTH][TILE_WIDTH];
__shared__ float M[TILE_WIDTH][TILE_WIDTH];

int k = ceil(1.0 * Channel * K * K / TILE_WIDTH);
float acc = 0.0;

for (int i = 0; i < k; ++i) {

    if (row < Map_out && i * TILE_WIDTH + tx < Channel*K*K) {
        N[ty][tx] = mask[(size_t) row * Channel*K*K + i * TILE_WIDTH + tx];
    } else N[ty][tx] = 0.0f;

    int tile_pos = i * TILE_WIDTH + ty;

    int c = tile_pos / (K*K);
    int p = (tile_pos % (K*K)) / K;
    int q = (tile_pos % (K*K)) % K;

    if (b < Batch && c < Channel && h < Height_out && w < Width_out) {
        M[ty][tx] = in_4d(b, c, h + p, w + q);
    } else M[ty][tx] = 0.0f;

    __syncthreads();

    if (row < Map_out && col < numRows) {
        for (int k = 0; k < TILE_WIDTH; k++) {

```

```

        acc += N[ty][k] * M[k][tx];
    }
}
__syncthreads();
}

if (row < Map_out && col < numRows) {
    output[b * Map_out * image_size + row * image_size + h * Width_out + w] = acc;
}

}

```

5.4 Analysis

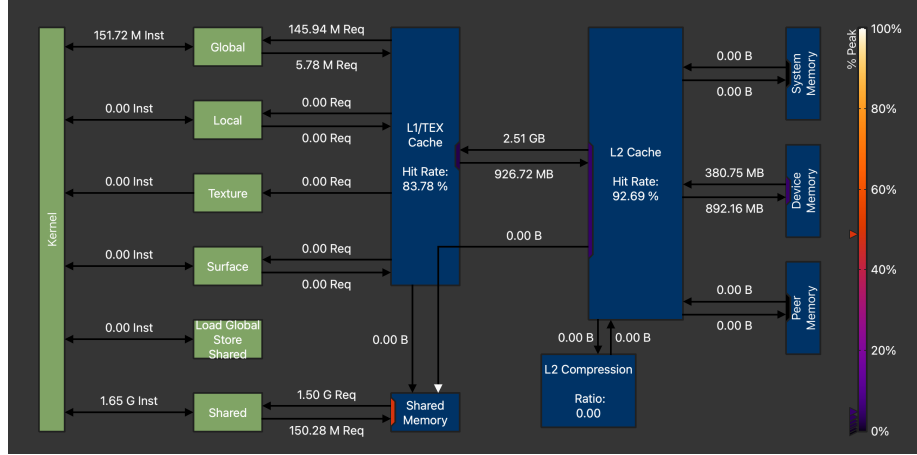


Figure 9: Memory Analysis: Req 2

We immediately note incredible performance improvements. From op times of near $200ms$ in PM2 to around $65ms$ for batch size $b = 10,000$. Additionally, we see that we are heavily utilizing shared memory, and minimizing our usage of global device memory. All of this is because we are working within one kernel, and not having to do global memcopies and data movement between kernels. We also have better cache performance as we are working in tiled chunks throughout. On the SOL Analysis, we see that our compute and memory throughput are close to 90% on both, indicating good GPU usage of both the compute and memory units.

The kernel fusion and dynamic unrolling optimization helped increase shared memory usage, increase cache hit rates, and overall decrease op times.

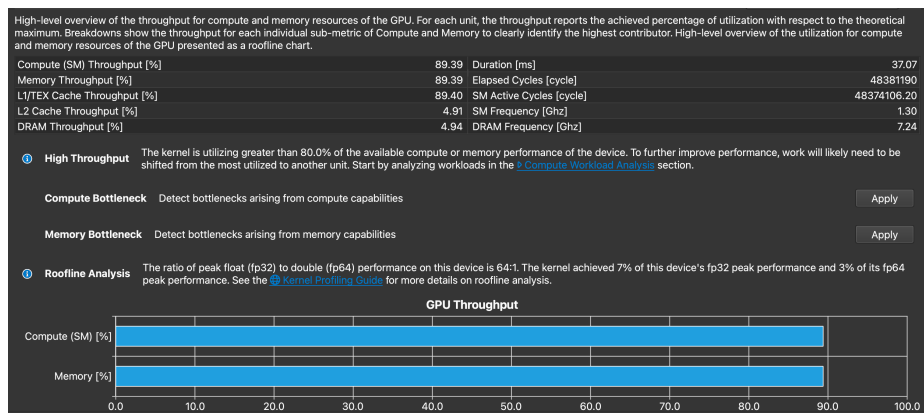


Figure 10: SOL Analysis: Req 2

5.5 References

1. Did not use references... I just spent some time at a white board drawing it out...

6 Op 0: Constant Memory

<https://drive.google.com/drive/folders/1B0AyuMM1-kQcSNsLKuqURaCuE4Qqkqyv?usp=sharing>

6.1 Motivation

If we have some data that we know never changes, and is "sufficiently" small, why not just cache it permanently somewhere? That is exactly what constant memory is. It is a bank of memory with similar access times as an L2 cache, but purely for data that is never modified after written to. This is helpful because it avoids CUDA to have to consider cache coherence and other factors, allowing a straightforward cache hit to the local memory.

6.2 Integration

Integration for constant memory is pretty straightforward. We just update the device mask to point to a constant memory allocated array. This means it synergizes very well for other optimizations.

6.3 Code

We note that the mask is relatively small, and continuously used, which makes sense to store it in constant memory. Constant memory has similar access times to the *L1* cache, which would improve the performance to our initial PM2 kernel, as we lessen the global memory requests.

We begin with the constant memory definition.

```
__constant__ float c_mask[10000]; // max mask dimensions
```

We then need to modify the *memcpy* to load into constant memory.

```
cudaMemcpyToSymbol(c_mask, host_mask, Map_out * Channel * K * K * sizeof(float));
```

Lastly, we modify the matrix multiply kernel to incorporate the mask. In our case, the mask is equivalent to the matrix *A* passed in as input. We can simply drop the following line in place of the load from *A* into shared memory.

```
tileA[ty][tx] = c_mask[(size_t) row * numAColumns + tileId * TILE_WIDTH + tx];
```

That's it! The mask is now loaded into constant memory.

6.4 Analysis

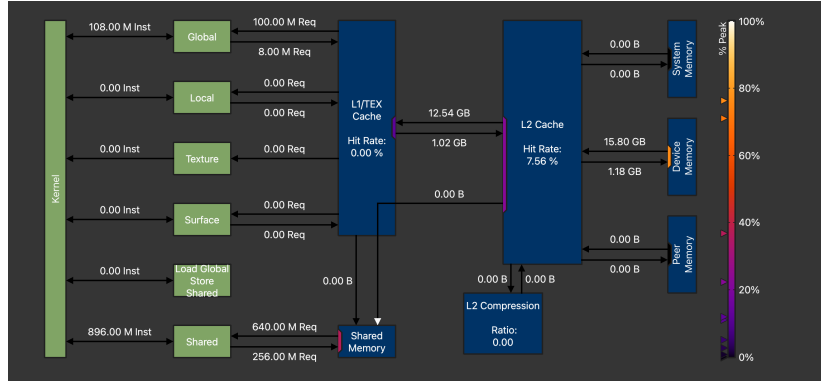


Figure 11: Nsight Compute Memory Analysis: Op 0

Though not a significant performance improvement, we see that we have less stress on the Device Memory. Rather than a peak of around 55% from PM2 baseline, we have reduced that to around 30%, indicating that we indeed are not requesting the global memory as much.

This makes sense because we don't request as much from global memory, as the mask is cached into constant memory and not global.

The difference is less drastic than I hoped, but the size of the mask is significantly (in fact, orders of magnitude) smaller than the other matrices involved. As such, it makes sense that the improvement from switching it to local is minor to the overall performance, as the runtime is dominated by the larger matrices.

6.5 References

1. Again... no references. Just changed a couple of declarations!
2. I suppose I did use https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__MEMORY.html, but that's just the memcpy to symbol docs

7 Op 1: Restrict

https://drive.google.com/drive/folders/1NTtdx4af7AZmhu5f6F1Tjo-cvJ60R24_?usp=sharing

7.1 Motivation

The nvcc compiler, and compilers in general, have to ensure code correctness above all else. When we have a function that may take multiple pointer arrays, e.g. `func(int * a, int * b)`, these compilers have no guarantee that *a* and *b* alias to the same array.

Thus, they cannot use any intermediary values to keep track, before writing back, as the two arrays may interfere with each other. However, by using the `__restrict__` keyword, we explicitly tell the compiler that these are non overlapping regions in memory, and that it may optimize as needed. For CUDA, another restriction is that the passed in pointer arrays must be declared *const*.

7.2 Integration

This is seamless to integrate. We just update the function headers for our kernels. Very well synergized with every optimization.

7.3 Code

We incorporate these keywords into our kernels as follows.

We modify the matrix multiplication kernel with the restrict operator.

```
__global__ void matrixMultiplyShared(const float * __restrict__ A,
                                     const float * __restrict__ B,
                                     float * C,
                                     int numRows, int numAColumns,
                                     int numBRows, int numBColumns,
                                     int numCRows, int numCColumns)
```

Similarly, we modify the permute kernel.

```
__global__ void matrix_permute_kernel(const float * __restrict__ input,
                                       float * output, int Map_out,
                                       int Batch, int image_size)
```

And the unrolled kernel.

```
__global__ void matrix_unrolling_kernel(const float* __restrict__ input, float* output,

                                         const int Batch, const int Channel,
                                         const int Height, const int Width,
                                         const int K)
```

With these minor modifications, we have allowed nvcc to make modifications and optimizations to our software, by assuming that the restricted arrays do not overlap or interfere with each other.

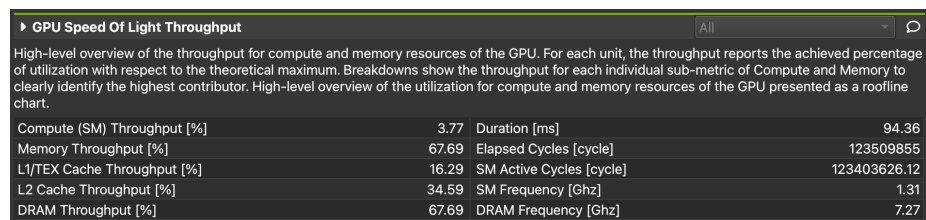
7.4 Analysis

Attached is the optime report.

```
Test batch size: 10000
Loading fashion-mnist data...Done
Loading model...Done
Conv-GPU==
Layer Time: 252.705 ms
Op Time: 83.472 ms
Conv-GPU==
Layer Time: 255.585 ms
Op Time: 127.207 ms
```

Test Accuracy: 0.8714

Op-times clearly didn't get better... let's examine the profiling results more closely.



High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	3.77	Duration [ms]	94.36
Memory Throughput [%]	67.69	Elapsed Cycles [cycle]	123509855
L1/TEX Cache Throughput [%]	16.29	SM Active Cycles [cycle]	123403626.12
L2 Cache Throughput [%]	34.59	SM Frequency [Ghz]	1.31
DRAM Throughput [%]	67.69	DRAM Frequency [Ghz]	7.27

Figure 12: SOL Analysis: Op 1

Seems to be lower SM throughput and worsened memory throughput. However, the L2 Cache *slightly* improved indicating some usage of the constant bank. One potential reason as to why the throughputs decreased is that the restrict operator allows nvcc to use local registers to hold temporary values better, and thus would require each thread to require more local memory. This would imply worse scheduling / warps having to wait.

7.5 References

1. <https://developer.nvidia.com/blog/cuda-pro-tip-optimize-pointer-aliasing/>

8 Op 4: cuBLAS

<https://drive.google.com/drive/folders/1Liwg0awUp-wRNW427TJ-a2PsbGquNByq?usp=sharing>

8.1 Motivation

Surely there must be a better way to do GEMM than have to program tensor cores every time right...? Well, cuBLAS is a framework provided by Nvidia just for that! It provides an API to do GEMM, utilizing the maximum capabilities of the GPU. It assumes matrices are column major by default, but allows us to do a lot of flexibility including transposing, specifying leading dimensions, etc. Let's try and incorporate it into our software.

8.2 Integration

As with tensor cores, we just need to change the matrix multiplication kernel. It's a drop in replacement for it.

8.3 Code

We create the handle and status variable that are needed to initialize cuBLAS.

```
cublasStatus_t status;  
cublasHandle_t handle;
```

And in the prologue, we can just initialize it

```
status = cublasCreate(&handle);
```

All we need to do now is call the GEMM function.

```
float alpha = 1.0f;  
float beta = 0.0f;  
cublasSgemv(handle, CUBLAS_OP_N, CUBLAS_OP_N,  
            Width_matmul, Height_matmul,  
            Width_conv_filters, &alpha,  
            unrolled_matrix, Width_unrolled,  
            device_mask, Width_conv_filters,  
            &beta, matmul_output, Width_matmul);  
cudaDeviceSynchronize();
```

Here we use a neat little trick. cuBLAS assumes column major matrices, which means our transposed matrices. We can use the fact that

$$(AB)^T = B^T A^T$$

But, $(AB)^T$ is just our *column-major* matrix transposed, which is row major! Exactly what we want. So we just swap the input as necessary and let cuBLAS handle the rest.

8.4 Analysis

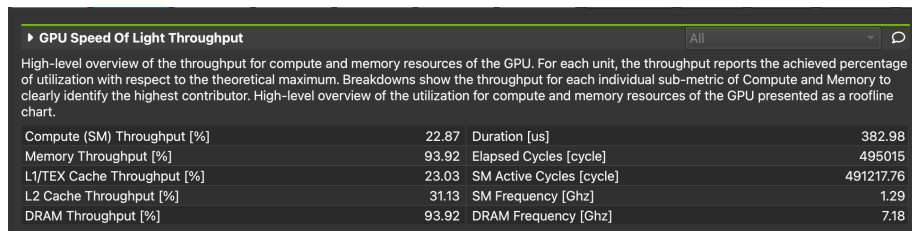


Figure 13: SOL Analysis: Req 4

cuBLAS seems to have split up the multiplication across many different kernels. We will just profile one of them. We see a much higher memory throughput for cuBLAS, primarily in terms of DRAM.

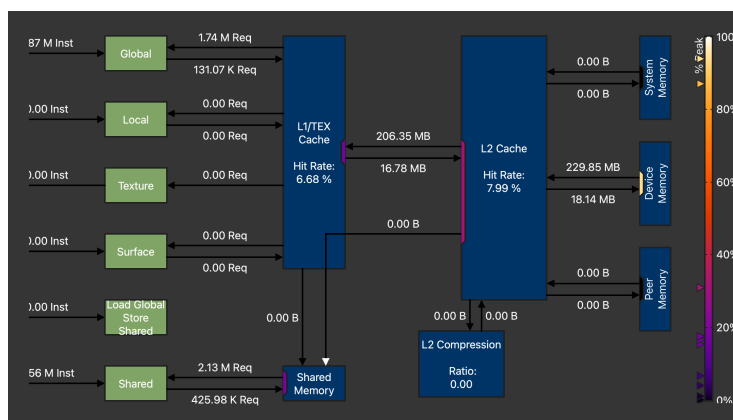


Figure 14: Memory Analysis: Op 4

We see much greater usage of the L2 cache than prior benchmarks on the matrix multiplication kernels. This is presumably because cuBLAS breaks the multiplication into smaller instances (thus the numerous profiled kernels/results). This means everything within the smaller instances has better spacial and temporal locality.

8.5 References

1. <https://docs.nvidia.com/cuda/cublas/index.html>
2. <https://docs.nvidia.com/cuda/cublas/index.html#cublas-level-3-function-reference>

9 Op 5: FP16

https://drive.google.com/drive/folders/1Aop-SFa2xG1jBIsz_GiKRgrmdmq60zZJ?usp=sharing

9.1 Code

One natural extension to making our code faster is to, well, use less data. If we initially had 32 bit floating points, we can half that to CUDA's built in *__half* type, and have much less computation.

Similarly to how we implemented the restrict operators, we just need to change types of our data and where we handle it. Namely, we will pass in *float* from the (and to the) user, but internally convert everything to half.

We modify the unrolling kernel to output a half, the matrix kernel to take in *B* as a half, and output *C* as a half, and the permute kernel to take the input as half.

Within the kernel, all we modify are when we load and store from shared memory. For example,

```
if (row < numRows && tileId * TILE_WIDTH + tx < numAColumns) {
    tileA[ty][tx] = __float2half(
        A[(size_t) row * numAColumns + tileId * TILE_WIDTH + tx]);
} else {
    tileA[ty][tx] = __float2half(0);
}
```

We then modify parts of the unroll and permute kernel as follows.

```
output[h_unroll * Batch * Height_out * Width_out + w_unroll] =
    __float2half(in_4d(b, c, h + p, w + q));

output[b * Map_out * image_size + m * image_size + x] =
    __half2float(input[m * Batch * image_size + b * image_size + x]);
```

Essentially, whenever we're converting to intermediate data, we use the half operator, and only when interfacing with user stuff do we convert back to float.

9.2 Analysis

The crux of the floating point 32 to floating point 16 optimization is to reduce the size of the data we are working with. This naturally lets us do more computations, and possibly even vectorize computations with the hardware. For example, if our hardware is capable of 32 bit operations, with 16 bit data types it may be possible to vectorize and stuff two computations into one.

We see a drastic improvement in op-times, from $> 192ms$ to $< 132ms$. However, this does come at a slight drop in accuracy. This is usually not a problem, as machine learning models can be trained to learn this error, and it hardly affects performance (albeit may require more training steps).

```
Test batch size: 10000
Loading fashion-mnist data...Done
Loading model...Done
Conv-GPU==
Layer Time: 244.134 ms
Op Time: 66.8456 ms
Conv-GPU==
Layer Time: 191.676 ms
Op Time: 64.6321 ms
```

Test Accuracy: 0.8716

I suppose... for completeness, here's the ncu profiler memory results.

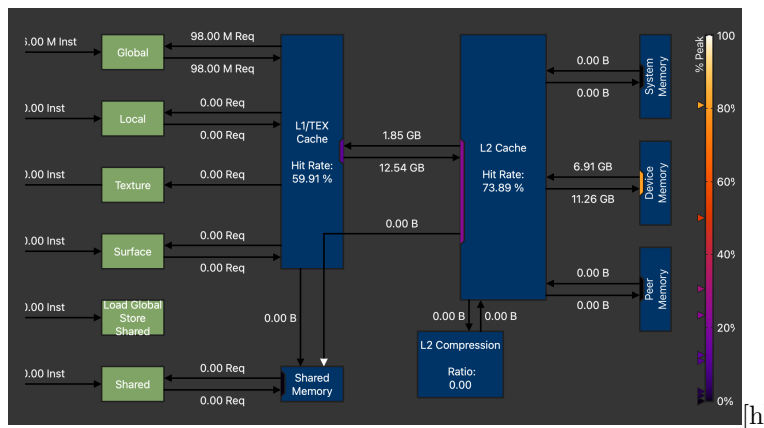


Figure 15: Memory Analysis: Op 5

As suspected, we're utilizing Device Memory less, and that's because our data types are 16 bit instead of 32 bit.

1. No references... just updates some *floats* to *half* and benchmarked

10 Conclusion

In PM3, we explored numerous different ways to optimize the convolution function involved in a CNN. We began with a pure CPU implementation, which had atrocious runtime. From there, we built a basic GPU implementation.

Then started the real fun. We swapped that out for a matrix multiplication, where we unrolled the computation and reduced it to a matrix multiplication. We introduced numerous different optimizations, leveraging different features CUDA provides: streams, tensor cores, etc. From there, we tried fusing kernels, and many other routes from minor compiler directives to trying cuBLAS.

In the end, we managed to get an op-time of *80ms* for batch= 10000, a massive improvement from the start!